

ASS-18.4

T.Shivani

2303A51312

BATCH – 05

TASK – 01

Prompt : Create a Python script using the requests library to fetch movie details from OMDb API. Requirements: Take movie title as user input, Fetch data from API using API key, Display: Title, Year, IMDb Rating, Plot, Handle errors: invalid movie, API key issues, network errors, empty response, Use clean formatting and functions

Code : import requests

API_KEY = "d69f7ff3"

BASE_URL = "http://www.omdbapi.com/"

def get_movie_details(movie_title):

 params = {

 "apikey": API_KEY,

 "t": movie_title

 }

 try:

 response = requests.get(BASE_URL, params=params)

 response.raise_for_status() # Raise an error for HTTP errors

 data = response.json()

 if data.get("Response") == "False":

 print("Movie not found.")

 return None

 return data

 except requests.RequestException as e:

 print(f"Error fetching movie details: {e}")

 return None

```

def display_movie_details(movie_data):
    if not movie_data:
        return

    print(f"Title: {movie_data.get('Title', 'N/A')}")
    print(f"Year: {movie_data.get('Year', 'N/A')}")
    print(f"IMDb Rating: {movie_data.get('imdbRating', 'N/A')}")
    print(f"Plot: {movie_data.get('Plot', 'N/A')}")

def main():
    movie_title = input("Enter the movie title: ")
    movie_data = get_movie_details(movie_title)
    display_movie_details(movie_data)

if __name__ == "__main__":
    main()

```

Output:

```

def main():
    movie_title = input("Enter the movie title: ")
    movie_data = get_movie_details(movie_title)
    display_movie_details(movie_data)
if __name__ == "__main__":
    main()

*** Enter the movie title: 4
Title: John Wick: Chapter 4
Year: 2023
IMDb Rating: 7.6
Plot: John Wick uncovers a path to defeating The High Table. But before he can earn his freedom, Wick must face off against a n

```

Explanation : This script allows users to fetch movie details from the OMDb API by entering a movie title and an API key. It uses the requests library to make an HTTP GET request to the API, retrieves the movie details in JSON format, and displays the title, year, IMDb rating, and plot. The script also includes error handling for invalid movie titles, API key issues, network errors, and empty responses.

TASK – 02

Prompt : Create a Python script using the requests library to fetch public transport arrival times from an API. Take stop ID as user input, validate it, and display the next 3 arrivals. Handle errors like invalid stop ID, server issues, timeout, and empty response using try-except. Include at least one retry attempt and keep the output clean and readable.

Code : import requests

```

API_KEY = "d69f7ff3"

BASE_URL = " http://www.omdbapi.com/"

def get_movie_details(movie_title):

    params = {

        "apikey": API_KEY,

        "t": movie_title

    }

    try:

        response = requests.get(BASE_URL, params=params)

        response.raise_for_status() # Raise an error for HTTP errors

        data = response.json()

        if data.get("Response") == "False":

            print("Movie not found.")

            return None

        return data

    except requests.RequestException as e:

        print(f"Error fetching movie details: {e}")

        return None

def display_movie_details(movie_data):

    if not movie_data:

        return

    print(f"Title: {movie_data.get('Title', 'N/A')}")

    print(f"Year: {movie_data.get('Year', 'N/A')}")

    print(f"IMDb Rating: {movie_data.get('imdbRating', 'N/A')}")

    print(f"Plot: {movie_data.get('Plot', 'N/A')}")

def main():

    movie_title = input("Enter the movie title: ")

    movie_data = get_movie_details(movie_title)

```

```
display_movie_details(movie_data)

if __name__ == "__main__":
    main()'''
```

#Create a Python script using the requests library to fetch public transport arrival times from an API. Take stop ID as user input, validate it, and display the next 3 arrivals. Handle errors like invalid stop ID, server issues, timeout, and empty response using try-except. Include at least one retry attempt and keep the output clean and readable.

#give code

```
import requests
```

```
import time
```

```
from datetime import datetime
```

```
API_URL = "https://api.tfl.gov.uk/StopPoint/{stop_id}/Arrivals"
```

```
def fetch_arrival_times(stop_id):
```

```
    url = API_URL.format(stop_id=stop_id)
```

```
    try:
```

```
        response = requests.get(url, timeout=5)
```

```
        response.raise_for_status()
```

```
        data = response.json()
```

```
        # TfL returns a list, not a dict
```

```
        if not isinstance(data, list) or len(data) == 0:
```

```
            print("⚠️ No arrivals found for this stop.")
```

```
            return None
```

```
# Sort by arrival time
```

```
data.sort(key=lambda x: x.get("timeToStation", 999999))
```

```
return data[:3] # next 3 arrivals
```

```
except requests.exceptions.HTTPError as e:
```

```
    print("❌ Invalid Stop ID or API issue:", e)
```

```
except requests.exceptions.ConnectionError:
```

```
    print("❌ Network error. Check your internet connection.")
```

```
except requests.exceptions.Timeout:
```

```
    print("❌ Request timed out.")
```

```
except requests.exceptions.RequestException as e:
```

```
    print("❌ Unexpected error:", e)
```

```
return None
```

```
def format_time(seconds):
```

```
    minutes = seconds // 60
```

```
    return f"{minutes} min"
```

```
def display_arrivals(arrivals):
```

```
    if not arrivals:
```

```
        print("⊘ No data to display.")
```

```
return
```

```
print("\n 🚌 Next Bus Arrivals")
```

```
print("-" * 40)
```

```
for bus in arrivals:
```

```
    line = bus.get("lineName", "N/A")
```

```
    destination = bus.get("destinationName", "Unknown")
```

```
    time_to_arrival = format_time(bus.get("timeToStation", 0))
```

```
    print(f" 🚌 Bus {line} → {destination}")
```

```
    print(f"    Arrival in: {time_to_arrival}")
```

```
    print("-" * 40)
```

```
def main():
```

```
    print(" 🟡 Welcome to Smart Bus Arrival Checker")
```

```
    stop_id = input("Enter TfL Stop ID (e.g., 490008660N): ").strip()
```

```
    if not stop_id:
```

```
        print(" ❌ Stop ID cannot be empty.")
```

```
        return
```

```
    arrivals = fetch_arrival_times(stop_id)
```

```
    if arrivals is None:
```

```
        print(" 🔄 Retrying in 2 seconds...")
```

```

time.sleep(2)

arrivals = fetch_arrival_times(stop_id)

display_arrivals(arrivals)

if __name__ == "__main__":
    main()

```

Output:

```

... Enter the movie title: 7
Title: Furious 7
Year: 2015
IMDb Rating: 7.1
Plot: Deckard Shaw seeks revenge against Dominic Toretto and his family for his comatose brother.
↑ Welcome to Smart Bus Arrival Checker
Enter Tfl Stop ID (e.g., 490008660N): 88
❌ Invalid Stop ID or API issue: 404 Client Error: Not Found for url: https://api.tfl.gov.uk/StopPoint/88/Arrivals
🔄 Retrying in 2 seconds...
❌ Invalid Stop ID or API issue: 404 Client Error: Not Found for url: https://api.tfl.gov.uk/StopPoint/88/Arrivals
🚫 No data to display.

```

Explanation : The program takes a stop ID as input and fetches arrival data from the TfL API using the requests library. It processes the response, sorts arrivals by nearest time, and displays the next 3 arrivals with details. It also handles errors (timeout, connection, invalid data) and includes a retry mechanism for reliability.

TASK – 03

Prompt : Create a Python script using the requests library to fetch live stock data from a financial API (such as Alpha Vantage). The program should accept a stock symbol as user input and display the current price, day high, and day low. It must handle errors like invalid stock symbols, API rate limits, and missing fields in the JSON response. Ensure proper validation and safe parsing of the

API response using try-except blocks, and display clean, user-friendly output without any crashes.

Code : import requests

API_KEY = "YOUR_API_KEY" # Replace with your Alpha Vantage API key

BASE_URL = "https://www.alphavantage.co/query"

```
def fetch_stock_data(symbol):  
    if not symbol.strip():  
        print("❌ Stock symbol cannot be empty.")  
        return None  
  
    params = {  
        "function": "GLOBAL_QUOTE",  
        "symbol": symbol.upper().strip(),  
        "apikey": API_KEY  
    }  
  
    try:  
        response = requests.get(BASE_URL, params=params, timeout=5)  
        response.raise_for_status()  
        data = response.json()  
  
        # Handle API rate limit  
        if "Note" in data:  
            print("⚠️ API limit reached. Please try again later.")  
            return None  
  
        # Handle invalid API key  
        if "Error Message" in data:  
            print("❌ Invalid API key or request.")  
            return None  
  
        quote = data.get("Global Quote")
```

if not quote:

print(" ❌ Invalid stock symbol or no data found.")

return None

return quote

except requests.exceptions.ConnectionError:

print(" ❌ Network error. Check your internet connection.")

except requests.exceptions.Timeout:

print(" ❌ Request timed out. Try again.")

except requests.exceptions.RequestException as e:

print(" ❌ Unexpected error:", e)

return None

def display_stock_data(quote):

try:

symbol = quote.get("01. symbol", "N/A")

price = quote.get("05. price", "N/A")

high = quote.get("03. high", "N/A")

low = quote.get("04. low", "N/A")

print("\n 📊 Stock Details")

print("-" * 40)

print(f"Symbol : {symbol}")

```
print(f"Current Price: ${price}")  
print(f"Day High   : ${high}")  
print(f"Day Low    : ${low}")  
print("-" * 40)
```

except Exception:

```
print("❌ Error displaying stock data (missing fields).")
```

def main():

```
print("💹 Live Stock Price Checker")
```

```
symbol = input("Enter stock symbol (e.g., AAPL, TSLA): ")
```

```
stock_data = fetch_stock_data(symbol)
```

if stock_data:

```
    display_stock_data(stock_data)
```

else:

```
    print("⚠️ Unable to retrieve stock data.")
```

if __name__ == "__main__":

```
    main()
```

output:

```
... Enter a stock symbol (e.g., AAPL, GOOGL): AAPL
```

```
--- Stock Data for AAPL ---
```

```
Current Price: $252.62
```

```
Day High:      $255.00
```

```
Day Low:       $251.60
```

```
-----
```

Explanation: The script takes a stock symbol as input and fetches its current price, day high, and day low from a financial API. It safely handles errors like invalid symbols, rate limits, and missing data, ensuring clean output without crashes.

TASK – 04:

Prompt : Generate a Python script that takes an IP address as input, validates its format, and uses a Geolocation API (like ip-api.com) to fetch country, city, and ISP. Include error handling for invalid IPs, API failures, and JSON parsing issues, and display results clearly.

Code : import requests

import re

API_URL = "http://ip-api.com/json/"

def is_valid_ip(ip):

"""Validate IPv4 address format"""

pattern = re.compile(r"^(?:[0-9]{1,3}\.){3}[0-9]{1,3}\$")

if not pattern.match(ip):

return False

Check each octet is between 0–255

parts = ip.split(".")

for part in parts:

if int(part) < 0 or int(part) > 255:

return False

```
return True
```

```
def fetch_ip_details(ip):
```

```
    """Fetch geolocation data from API"""
```

```
    try:
```

```
        url = API_URL + ip
```

```
        response = requests.get(url, timeout=5)
```

```
        response.raise_for_status()
```

```
    try:
```

```
        data = response.json()
```

```
    except ValueError:
```

```
        print("❌ Error parsing server response (invalid JSON).")
```

```
        return None
```

```
    # API-specific error handling
```

```
    if data.get("status") == "fail":
```

```
        print(f"❌ API Error: {data.get('message', 'Invalid IP address')}")
```

```
        return None
```

```
    return data
```

```
except requests.exceptions.ConnectionError:
```

```
    print("❌ Network error. Please check your internet connection.")
```

```
except requests.exceptions.Timeout:
```

```
    print("❌ Request timed out. Try again later.")
```

```
except requests.exceptions.HTTPError as e:
```

```
    print("❌ HTTP error:", e)
```

```
except requests.exceptions.RequestException as e:
```

```
    print("❌ Unexpected error:", e)
```

```
return None
```

```
def display_ip_details(data):
```

```
    """Display results cleanly"""
```

```
    if not data:
```

```
        print("⚠️ No data to display.")
```

```
        return
```

```
    print("\n🌐 IP Geolocation Details")
```

```
    print("-" * 40)
```

```
    print(f"IP Address : {data.get('query', 'N/A')}")
```

```
    print(f"Country   : {data.get('country', 'N/A')}")
```

```
    print(f"City       : {data.get('city', 'N/A')}")
```

```
    print(f"ISP        : {data.get('isp', 'N/A')}")
```

```
    print("-" * 40)
```

```
def main():
```

```
    print("🌐 IP Geolocation Finder")
```

```

ip = input("Enter an IP address: ").strip()

if not is_valid_ip(ip):
    print("❌ Invalid IP address format.")
    return

data = fetch_ip_details(ip)

if data:
    display_ip_details(data)
else:
    print("⚠️ Could not retrieve IP details.")

```

```

if __name__ == "__main__":
    main()

```

Output :

```

... Enter an IP address (e.g., 8.8.8.8, 203.0.113.45): 8.8.8.8

Attempting to fetch geolocation data for IP: 8.8.8.8...
--- Geolocation Data for 8.8.8.8 ---
Country: United States
City:    Ashburn
ISP:     Google LLC
-----

```

Explanation :

This script takes an IP address, checks if it is valid, and then calls a Geolocation API to get details like country, city, and ISP. It also handles errors such as invalid IP format, API failure, and incorrect JSON response to ensure the program runs safely.

TASK – 05

Prompt : Generate a Python script that simulates payment processing using a sandbox Payment API. It should send a POST request with amount, currency, and payment method, validate inputs, handle errors (invalid details, HTTP errors, network issues), display transaction status, and log failed transactions to a file.

Code : import requests

import json

from datetime import datetime

Example sandbox endpoint (mock API)

API_URL = "https://jsonplaceholder.typicode.com/posts"

def validate_payment(amount, currency, method):

if amount <= 0:

print("❌ Amount must be greater than 0.")

return False

if not currency.isalpha() or len(currency) != 3:

print("❌ Currency must be a 3-letter code (e.g., USD, INR).")

return False

valid_methods = ["card", "upi", "netbanking"]

if method.lower() not in valid_methods:

print(f"❌ Invalid payment method. Choose from {valid_methods}")

return False

return True

```

def log_failed_transaction(data, error_message):
    with open("failed_transactions.log", "a") as file:
        log_entry = {
            "timestamp": str(datetime.now()),
            "data": data,
            "error": error_message
        }
        file.write(json.dumps(log_entry) + "\n")


def process_payment(amount, currency, method):
    payload = {
        "amount": amount,
        "currency": currency.upper(),
        "payment_method": method.lower()
    }

    try:
        response = requests.post(API_URL, json=payload, timeout=5)
        response.raise_for_status()

        try:
            data = response.json()
        except ValueError:
            print("❌ Invalid JSON response from server.")
            log_failed_transaction(payload, "JSON parsing error")
            return None

```

```
return data
```

```
except requests.exceptions.HTTPError as e:
```

```
    print("❌ HTTP error:", e)
```

```
    log_failed_transaction(payload, str(e))
```

```
except requests.exceptions.ConnectionError:
```

```
    print("❌ Network error. Check your connection.")
```

```
    log_failed_transaction(payload, "Connection error")
```

```
except requests.exceptions.Timeout:
```

```
    print("❌ Request timed out.")
```

```
    log_failed_transaction(payload, "Timeout")
```

```
except requests.exceptions.RequestException as e:
```

```
    print("❌ Unexpected error:", e)
```

```
    log_failed_transaction(payload, str(e))
```

```
return None
```

```
def display_result(response_data):
```

```
    if not response_data:
```

```
        print("⚠️ Payment failed.")
```

```
        return
```

```
print("\n💳 Payment Status")
```

```
print("-" * 40)

print("Status    : SUCCESS )

print(f"Transaction ID: {response_data.get('id', 'N/A')}")

print("-" * 40)
```

```
def main():
```

```
    print("💰 Payment Processing Simulator")
```

```
    try:
```

```
        amount = float(input("Enter amount: "))
```

```
    except ValueError:
```

```
        print("❌ Invalid amount.")
```

```
        return
```

```
    currency = input("Enter currency (e.g., USD, INR): ").strip()
```

```
    method = input("Enter payment method (card/upi/netbanking): ").strip()
```

```
    if not validate_payment(amount, currency, method):
```

```
        return
```

```
    response = process_payment(amount, currency, method)
```

```
    display_result(response)
```

```
if __name__ == "__main__":
```

```
    main()
```

output:

```
...
--- Enter Payment Details ---
Enter amount (e.g., 50.00): 50.00
Enter currency (e.g., USD, EUR, GBP, JPY): USD
Enter payment method (credit_card, paypal, bank_transfer): paypal

Attempting to process payment for 50.0 USD via paypal...
Transaction Status: SUCCESS
Transaction ID: txn_e0484573
Message: Payment processed successfully.

Transaction processing completed.
Log file 'failed_transactions.log' does not exist yet (no failed transactions).
```

Explanation : This script sends payment details (amount, currency, method) to a sandbox API, checks if inputs are valid, and processes the transaction. It handles errors like invalid data, server issues, or network failures, shows the payment status, and logs failed transactions to a file.